NSSF A3 Classifier Pipeline

Technical Documentation

Pipeline Flow • Algorithm Reference • Model Fine-Tuning Guide

Model	Backbone	Classes	Runtime
EfficientNet-B0	**PyTorch + ONNX**	**A3 / Other**	**GPU (T4) / CPU**

1. Overview

The NSSF A3 Classifier Pipeline is an end-to-end machine-learning system that identifies whether a scanned document is printed on A3-sized paper or on some other size (A4, Letter/Legal, or other). It combines a fast deterministic algorithm (Algorithm v6) with a fine-tuned deep-learning model (EfficientNet-B0) in a hybrid decision system that is robust to variable scan quality, missing metadata, and non-standard DPI settings.

NOTE: ⚠ The sort algorithm has an accuracy of upto 90%, it can perform better than the model. But the combination hybrid can achieve a higher accuracy, that's only if the model is tuned perfectly, and the parameters are tweaked.

1.1 Why a Hybrid Approach?

Neither a pure algorithm nor a pure neural network is sufficient alone:

- **Algorithm alone:** The algorithm is fast, transparent, and interpretable — but it degrades when image DPI metadata is missing or incorrect.
- **CNN alone:** A neural network generalises well across scan quality but can be fooled by images that look like A3 but are not.
- **Hybrid:** Combining both gives the confidence of metadata-based checks and the generalisation of learned features.

1.2 High-Level Architecture

At inference time the hybrid classifier runs two independent checks and resolves conflicts:

Stage	Component	Output
Preprocessing	Orientation detection (Tesseract OSD)	Rotation-corrected image

Stage	Component	Output
Branch A — Algorithm v6	DPI metadata + pixel-dimension heuristic	Label + confidence (high/low)
Branch B — EfficientNet-B0 (ONNX)	Visual classification via CNN	P(A3) probability 0–1
Fusion / Decision	Conflict-resolution rules (see §4.4)	Final label + method tag

2. Pipeline Steps — Detailed Flow

Run the notebook cells strictly top-to-bottom. Each step auto-skips if its output already exists on disk.

1 Install Dependencies

Installs Python packages and system tools needed by all subsequent steps.

```
pip install -q pillow pytesseract gdown py7zr onnx onnxruntime onnxscript
apt-get install -q tesseract-ocr
```

Key libraries and their roles:

Library	Role
pillow	Image loading, rotation, saving
pytesseract	Orientation detection (OSD) via Tesseract OCR
gdown	Google Drive download
py7zr	Extraction of .7z archives
torch / torchvision	EfficientNet-B0 training
onnx / onnxruntime	Model export and CPU/GPU inference
sklearn	Metrics: F1, confusion matrix, ROC-AUC

2 Download from Google Drive

Downloads the raw image archive using gdown.

Edit the FILES dictionary in the cell to point to your archive:

```
FILES = {
    '<YOUR_DRIVE_FILE_ID>': '/content/nssfA3_.7z',
}
```

⚠ The Google Drive file must be shared as 'Anyone with the link can view'. Find the file ID in the share URL: drive.google.com/file/d/<FILE_ID>/view

3 Extract Archive

Detects archive format by magic bytes and extracts into /content/nssfA3_.

The smart_extract() function handles .7z, .zip, and .tar.gz transparently and also detects single-folder wrapper archives, unwrapping them automatically. Supported image extensions: .jpg, .jpeg, .png, .tif, .tiff.

If extraction fails, verify that py7zr is installed and that the archive is not password-protected.

4 Algorithm Sort — Create Training Labels

Runs Algorithm v6 on all images and copies them into Sorted_A3 or Sorted_Other. These folders become the ground-truth labels for training.

This is the most important labelling step. The algorithm classifies each image without any learned model, purely from image metadata and pixel dimensions. The sorted results form the supervised training dataset.

Algorithm v6 — Classification Logic

The algorithm follows two branches depending on whether trustworthy DPI metadata is present:

Branch	Condition	Method
A	DPI metadata ≥ 200 (DPI_TRUST)	Convert short pixel dimension to mm using actual DPI. Compare to A3 (297mm) ± TOLERANCE_MM.
B	DPI metadata missing or < 200	Reverse-engineer implied DPI from pixel width. Score how close it is to a standard DPI (96, 100, 120, 150, 200, 300, 400, 600).

Confidence Levels

- diff < 60% of TOLERANCE_MM → high confidence
- diff ≥ 60% of TOLERANCE_MM → low confidence
- Branch B: implied DPI is within 8 of a standard DPI → high confidence

5 Build Train / Val / Test Splits

Copies images from Sorted_A3 and Sorted_Other into a folder structure that torchvision ImageFolder expects.

Default split ratios:

Split	Ratio	Purpose
Train	0.70	Model weight updates
Validation	0.15	Checkpoint selection & threshold tuning
Test	0.15	Final unbiased evaluation

6 DataLoaders + EfficientNet-B0 Model

Builds augmented data pipelines and defines the neural network architecture.

Training Augmentations

Transform	Parameters	Rationale
RandomHorizontalFlip	p=0.3	Documents may be flipped during scanning
RandomRotation	±4°	Slight skew from scanner alignment
ColorJitter	brightness & contrast ±0.35	Variable scanner exposure settings
RandomAffine	translate ±2%	Page not perfectly centred on scanner

EfficientNet-B0 Architecture

The model uses ImageNet pretrained weights with selective fine-tuning:

- All backbone layers frozen except the last 3 feature blocks
- Custom 2-layer classifier head: 1280 → 512 (SiLU) → 2 (logits)
- Dropout rates: 0.35 (after pooling), 0.20 (between linear layers)
- Approximately 10–15% of total parameters are trainable

7 Training Loop

Runs up to 30 epochs with early stopping, cosine LR schedule, and class-balanced sampling.

ℹ Requires a GPU runtime. Go to Runtime → Change runtime type → T4 GPU before starting.

Training Configuration Defaults

Parameter	Default	Description
EPOCHS	30	Maximum number of training epochs
LR	3e-4	Initial learning rate (AdamW)
WEIGHT_DECAY	1e-4	L2 regularisation
BATCH_SIZE	32	Images per gradient step
IMG_SIZE	240	Resize target (px); matches EfficientNet-B0 native
PATIENCE	6	Early-stop if val F1 does not improve for N epochs
label_smoothing	0.05	Prevents overconfident predictions

8 Threshold Tuning

Searches the decision threshold that maximises F1 on the validation set.

The model outputs logits that are converted to P(A3) via softmax. Instead of the default 0.5 cut-off, the pipeline searches thresholds in [0.10, 0.90] and saves the best to best_threshold.json.

ℹ Increasing the threshold raises precision (fewer non-A3 pages sent to the A3 folder). Decreasing it raises recall (fewer real A3 pages missed). Choose based on your business cost of each error type.

9 Test Set Evaluation

Reports classification metrics on the held-out test set using the tuned threshold.

Metrics reported: Precision, Recall, F1-score per class; Confusion Matrix; ROC-AUC. A correctly trained model should achieve ≥ 0.92 F1 on the A3 class.

10 ONNX Export

Serialises the trained model to ONNX format with dynamic batch dimension.

ONNX Runtime typically delivers 3–5× faster CPU inference than native PyTorch. The export uses opset 18. A speed benchmark (50 forward passes) is printed after export.

11 Hybrid Classifier + Final Batch Sort

Runs every source image through the full hybrid pipeline and sorts into Final_A3, Final_Other, or Final_Review.

Decision Fusion Rules

Method Tag	Condition	Action
both_agree	Algo and CNN agree on label	Accept the agreed label — highest confidence path
algo_high	Algo=high confidence, CNN disagrees	Flag for manual review (review_required)
cnn_override	Algo=low confidence, CNN disagrees	CNN label wins

12 Active Learning — Retrain with Corrections

Iterative improvement cycle: correct Final_Review, rebuild splits, fine-tune model.

Expected improvement per correction cycle:

- Baseline (algorithm labels only): ~90–94% F1

- After cycle 1 with 50+ corrections: +2–4% F1, review count drops ~50%
- After cycle 2 with 50+ corrections: +1–2% F1, review count → near zero

3. Full Parameter Reference

3.1 Algorithm v6 Parameters

Constant	Default	Effect of Changing
TOLERANCE_MM	18	Maximum mm deviation from A3 short edge (297mm) before labelling as Other. Increase to catch more A3 images at the cost of more false positives.
DPI_TRUST	200	Minimum DPI metadata value to use Branch A. Lowering it makes Branch A apply more often (faster, but less safe on unreliable metadata).
DPI_MIN	50	Absolute minimum implied DPI considered plausible.
DPI_MAX	1200	Absolute maximum implied DPI considered plausible.
STD_DPIS	[96,100,120,150,200,300,400,600]	Standard DPI values used by Branch B scoring. Add values if you see scans at other common DPIs in your dataset.

3.2 Model Training Parameters

Parameter	Default	Tuning Guidance
IMG_SIZE	240	Must match EfficientNet-B0 native resolution. Increase to 260 (B1) or 300 (B3) if you upgrade the backbone.
BATCH_SIZE	32	Reduce to 16 if GPU OOM. Increase to 64 on A100 for faster training.
EPOCHS	30	Usually stops early around epoch 15–20. Increase if training is still improving when patience triggers.
LR	3e-4	Starting LR. Reduce to 1e-4 if loss oscillates. Increase to 5e-4 for very large datasets.
WEIGHT_DECAY	1e-4	L2 regularisation. Increase to 5e-4 if overfitting (val loss rises while train loss falls).
TRAIN_RATIO	0.70	Fraction of labelled data used for training. Increase to 0.80 for small datasets.
VAL_RATIO	0.15	Fraction for validation and threshold tuning.
PATIENCE	6	Increase to 10 for more stable convergence; decrease to 4 to save GPU time.
SEED	42	Change to run a different random split; use multiple seeds to estimate variance.

4. Updating the Algorithm (v6 → v7+)

The algorithm lives in the algorithm_classify() function (Step 4). To update it without touching the model:

4.1 Adding a New Paper Size

Append to PAPER_SIZES. The tuple is (label_string, short_edge_in_mm):

```
PAPER_SIZES = [
    ('A3',          297.0),
    ('A4',          210.0),
    ('Letter/Legal',215.9),
    ('A2',          420.0),   # ← new
]
```

⚠ You must also update the classifier head output size from 2 to 3+ classes and re-train the model when adding new classes.

4.2 Tightening or Loosening the Tolerance

TOLERANCE_MM controls how far a measured short-edge can be from the nominal value and still be accepted:

- Reduce TOLERANCE_MM (e.g., 12) → fewer false positives, but misses slightly mis-sized scans
- Increase TOLERANCE_MM (e.g. 24) → more A3 images captured, but risks labelling A4 as A3 near the boundary

4.3 Adjusting DPI Trust Threshold

If your scanner always writes accurate metadata (e.g., Fujitsu or Canon enterprise models), reduce DPI_TRUST to 100 so Branch A applies more often:

```
DPI_TRUST = 100   # was 200
```

If metadata is often missing or wrong (consumer flatbeds, mobile scanning apps), keep it at 200 or raise it to 300.

4.4 Adding a New Standard DPI

If your scans are consistently captured at an unusual DPI (e.g., 400):

```
STD_DPIS = [96, 100, 120, 150, 200, 300, 400, 600]   # 400 already included
```

⚠ After any algorithm change, delete Sorted_A3 and Sorted_Other and re-run Step 4 to regenerate training labels.

4.5 Improving Orientation Detection

The algorithm calls Tesseract OSD to detect rotation. If orientation is frequently wrong (seen in debug output as unexpected rotate values), you can:

- Increase the minimum OSD confidence: change >= 2.0 to >= 3.5 in _get_rotation()
- Disable OSD entirely (return 0 always) if your scans are always upright
- Replace with a faster rotation estimator (e.g. scipy image moments)

5. Fine-Tuning the Model

5.1 When to Fine-Tune

- After correcting the Final_Review folder (active learning cycle — see Step 12)
- When F1 on new production data drops below acceptable levels
- When a new document scanner/scan profile is introduced that looks visually different
- When the class balance shifts significantly (e.g., a new batch is 95% A3)

5.2 Quick Fine-Tune (Active Learning Cycle)

This is the standard workflow after manually reviewing and correcting images:

```
# 1. Move corrected images:
#    Final_Review/A3_files   → Sorted_A3/
#    Final_Review/Other_files → Sorted_Other/

# 2. Delete old splits to force rebuild:
!rm -rf /content/dataset

# 3. Re-run Step 5 (builds new splits including corrections)

# 4. Load the best checkpoint and set a low LR:
model.load_state_dict(torch.load(MODEL_PT))
for p in model.parameters():
    p.requires_grad = True   # unfreeze all layers

# 5. In Step 7 header, set:
LR     = 5e-5   # was 3e-4
EPOCHS = 10     # shorter run

# 6. Re-run Steps 6 (DataLoaders), 7, 8, 9, 10
```

5.3 Upgrading the Backbone

To use a more powerful backbone (e.g., EfficientNet-B3 for higher accuracy):

```
def build_model():
    m = models.efficientnet_b3(weights='IMAGENET1K_V1')
    for p in m.parameters(): p.requires_grad = False
    for layer in list(m.features.children())[-3:]:
        for p in layer.parameters(): p.requires_grad = True
    in_f = m.classifier[1].in_features   # 1536 for B3
    m.classifier = nn.Sequential(
        nn.Dropout(p=0.35, inplace=True),
        nn.Linear(in_f, 512),
        nn.SiLU(),
        nn.Dropout(p=0.2),
        nn.Linear(512, 2),
    )
    return m.to(DEVICE)

# Also update:
IMG_SIZE = 300   # B3 native resolution
```

⚠ Larger backbones require more GPU memory. With B3 reduce BATCH_SIZE to 16 on a T4 GPU.

5.4 Hyperparameter Tuning Recipes

If Validation F1 Stagnates

- Try LR = 1e-4 with PATIENCE = 10
- Unfreeze more layers: change [-3:] to [-5:] in build_model()
- Add stronger augmentation: increase ColorJitter brightness to 0.5

If Model Overfits (train F1 >> val F1)

- Increase Dropout: change p=0.35 to p=0.5
- Increase WEIGHT_DECAY from 1e-4 to 5e-4
- Reduce the number of unfrozen layers

If Precision is Too Low (too many non-A3 pages sent to A3 folder)

- Increase the decision threshold in best_threshold.json (e.g. 0.65)
- Target a higher precision during threshold tuning by editing the threshold search to maximise precision @ recall ≥ 0.90

If Recall is Too Low (real A3 pages being missed)

- Decrease the threshold (e.g. 0.35)
- Collect more A3 training examples and re-train

5.5 Exporting After Fine-Tuning

After any model update, always re-run Steps 8 (threshold tuning), 9 (evaluation), and 10 (ONNX export) to ensure the deployed artefacts reflect the new weights.

```
# Verify new model is better than previous:
# Compare ROC-AUC from Step 9 before overwriting MODEL_ONNX
```

6. Output Files Reference

Path	Description
/content/Sorted_A3/	Images labelled as A3 by Algorithm v6 (training source)
/content/Sorted_Other/	Images labelled as Other by Algorithm v6 (training source)
/content/algo_report.csv	Per-image algorithm results: label, confidence, rotation
/content/dataset/	Structured train/val/test folder hierarchy
/content/nssf_efficientnet.pt	Best PyTorch model checkpoint (highest val F1)
/content/best_threshold.json	Optimal decision threshold {"threshold": 0.xx}
/content/nssf_efficientnet.onnx	Deployable ONNX model (use this in production)
/content/training_curves.png	Loss, F1, precision, and recall curves over epochs
/content/threshold_curve.png	Precision/Recall/F1 vs threshold sweep plot
/content/Final_A3/	Final sort output: confirmed A3 images
/content/Final_Other/	Final sort output: confirmed non-A3 images
/content/Final_Review/	Conflict cases requiring manual review
/content/final_report.csv	Per-image hybrid classifier results with method tags

7. Troubleshooting

Symptom	Resolution
Sorted_A3 is empty after Step 4	Check SOURCE_DIR. Run find_image_root() and print the result. Ensure images have supported extensions (.jpg, .png, .tif).
CUDA out of memory during training	Reduce BATCH_SIZE to 16. If still failing, set IMG_SIZE=224.
Val F1 stuck near 0.5	Class imbalance not handled — verify WeightedRandomSampler is active. Print class_counts.
ONNX export fails with dynamic_shapes error	Fallback to static shape export is automatic. No action needed.

Symptom	Resolution
final_report.csv shows high review_required count	Algo and CNN strongly disagree on many images. Check TOLERANCE_MM — may need tightening. Run one active learning cycle.
OSD rotation always 0 but images are rotated	OSD confidence threshold (2.0) may be too strict. Lower to 1.0 in _get_rotation(). Or pre-rotate images before feeding the pipeline.
Training curves show val loss increasing from epoch 5	Overfitting. Increase WEIGHT_DECAY, add more augmentation, or reduce unfrozen layers.

8. Recommended Improvement Roadmap

Priority	Improvement	How to Implement
High	**Active learning cycle 1**	Correct 50+ images in Final_Review → re-run Steps 5, 7–10 with LR=5e-5
High	**Increase training data**	Add more diverse scans from different scanner models to Sorted_A3 and Sorted_Other
Medium	**Upgrade to EfficientNet-B3**	Change build_model() backbone and set IMG_SIZE=300 (see §5.3)
Medium	**Multi-scale TTA at inference**	Average predictions at 3 scales (224, 240, 256) in onnx_classify()
Medium	**Expand to A2 / B4 detection**	Add paper sizes to PAPER_SIZES, change output classes to 4, re-train
Low	**Replace Tesseract OSD**	Use a lightweight CNN trained solely on page orientation (faster, no apt dep)
Low	**Batch ONNX inference**	Replace single-image calls with batched ONNX session for 8–16 images at once

— End of Documentation —